EDGE-TO-CLOUD BEARING FAILURE PREDICTION AND DIGITAL TWIN MONITORING USING INDUSTRIAL IoT

CO326 Course Project Report

Submitted by:

Group 21
E/20/062 Supun Dhananji
E/20/271 Hasini Nirmani
E/20/367 Binuri Senavirathna
E/20/370 Rashmika Sewmini

Date:
01.05.2026

Abstract

Bearing failures are a major cause of unplanned downtime in industrial rotating machinery, making early fault detection essential for predictive maintenance. This project presents an Edge-to-Cloud Industrial Internet of Things (IIoT) system for bearing failure prediction using vibration monitoring, edge intelligence, MQTT communication, and digital twin visualization.

The system simulates realistic bearing vibration behavior including normal operating vibrations, random noise, transient impact spikes, and progressive degradation representing gradual bearing wear. Sensor data is generated through a Python-based simulation and transmitted through an MQTT broker to an edge intelligence module.

Anomaly detection is performed using a hybrid approach combining threshold-based detection and statistical z-score analysis using a rolling window baseline. Abnormal operating conditions are classified into Normal, Warning, and Critical states and corresponding alerts are published through MQTT topics.

A Dockerized architecture integrating Mosquitto, Python edge processing and Node-RED provides an edge-to-cloud monitoring pipeline. A digital twin dashboard visualizes vibration trends, system health, and AI-generated fault alerts in real time.

The project demonstrates predictive maintenance principles, cyber-physical integration, anomaly detection at the edge, and Remaining Useful Life oriented monitoring using a scalable industrial IoT architecture.

Keywords: Bearing Failure Prediction, Edge AI, Digital Twin, Industrial IoT, MQTT, Predictive Maintenance.

1 Introduction

1.1 Background

Modern industrial systems are increasingly moving from reactive maintenance strategies toward predictive and condition-based maintenance approaches to improve reliability, reduce downtime and minimize operational costs.

Traditional maintenance strategies often repair equipment only after failures occur, resulting in unexpected shutdowns and productivity losses. Predictive maintenance overcomes these limitations by continuously monitoring machine condition and identifying faults before catastrophic failure.

Industrial Internet of Things (IIoT) technologies make this possible through integration of:

1. Sensors
2. Edge intelligence
3. Communication networks
4. Cloud platforms
5. Digital twin systems

Among industrial assets, bearings are critical components used in motors, pumps, turbines and rotating machinery. Their degradation often produces measurable changes in vibration signatures, making vibration analysis one of the most effective condition monitoring techniques.

1.2 Problem Motivation

Unexpected bearing failures may lead to:

1. Production downtime
2. Equipment damage
3. Increased maintenance costs
4. Safety risks

Therefore, a monitoring system capable of detecting abnormal vibration patterns and identifying progressive wear can support predictive maintenance and improve reliability.

1.3 Project Overview

This project develops an industrial Internet of Things (IIoT) based system for bearing failure prediction using vibration monitoring, edge intelligence and digital twin visualization.

The system simulates realistic bearing behavior, including:

- Normal operating vibrations

- Random noise
- Transient fault spikes
- Progressive degradation representing gradual bearing wear

Sensor data is generated through a Python-based simulation and transmitted through an MQTT broker to an edge intelligence module.

Anomaly detection is performed using a hybrid approach combining threshold-based detection and statistical z-score analysis using a rolling window baseline. Detected conditions are classified into Normal, Warning and Critical states, and corresponding alerts are published through MQTT topics.

A Dockerized architecture integrating Mosquitto, Python edge processing and Node-RED provides an edge-to-cloud monitoring pipeline. A digital twin dashboard visualizes vibration trends, system health and AI-generated alerts in real time.

The system implements the following key components:

- **Simulated vibration sensing** – generates realistic vibration signals with degradation, noise and impact spikes.
- **Edge anomaly detection** – detects abnormalities using thresholding and z-score analysis.
- **MQTT communication** – enables lightweight and reliable industrial data transport.
- **Digital twin dashboard visualization** – provides real-time monitoring of vibration trends and system health.
- **Trend-based Remaining Useful Life monitoring** – estimates failure progression based on vibration trend analysis.

The project follows industrial digital twin architecture principles and demonstrates cyber-physical integration using a containerized IIoT environment.

1.4 Evolution of Maintenance Strategies

Industrial maintenance strategies have evolved significantly over time. Initially, maintenance was purely reactive, where repairs were performed only after failures occurred. This later evolved into preventive maintenance based on scheduled servicing, and more recently into predictive maintenance using condition monitoring and analytics.

The emergence of digital twin technologies extends predictive maintenance further by enabling intelligent monitoring, simulation and optimization of maintenance decisions.

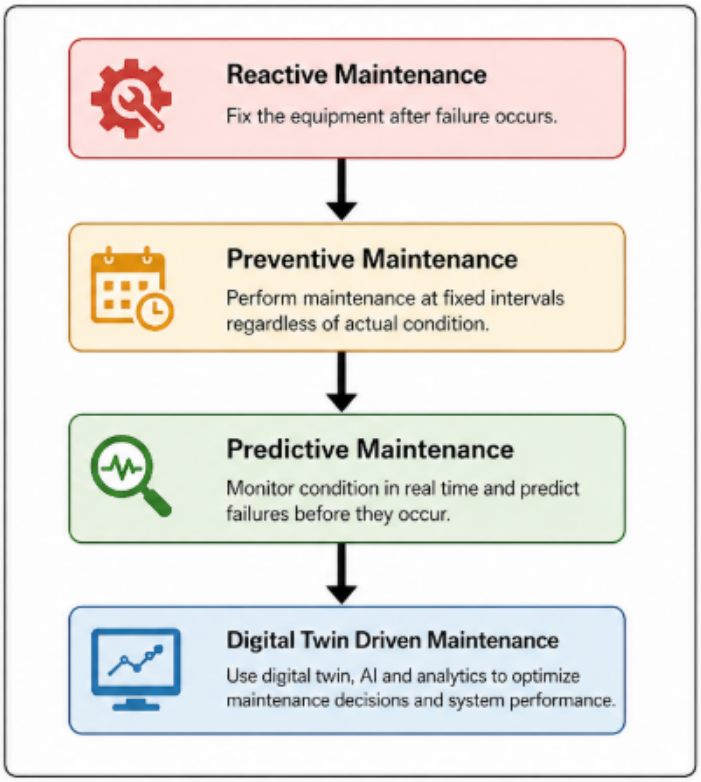

Figure 1: Evolution from Reactive to Digital Twin Driven Predictive Maintenance

As shown in Figure 1, modern digital twin driven maintenance integrates monitoring, analytics and cyber-physical synchronization to support intelligent predictive maintenance.

2 Objectives

2.1 Main Objective

To design and implement an Industrial IoT based edge-to-cloud bearing monitoring system capable of anomaly detection and digital twin visualization for predictive maintenance.

2.2 Specific Objectives

The specific objectives of the project are:

1. Develop a simulated vibration sensing model representing gradual bearing wear.
2. Implement edge intelligence anomaly detection using thresholding and statistical methods.
3. Establish MQTT-based communication for industrial data transport.
4. Develop a digital twin dashboard for real-time condition monitoring.
5. Demonstrate degradation trend monitoring for Remaining Useful Life estimation.
6. Implement the system using Dockerized industrial software components.

3 Problem Statement

Bearings undergo gradual wear during operation and often exhibit increasing vibration signatures before reaching failure conditions. Detecting these patterns early is crucial for predictive maintenance.

The challenge addressed in this project is to design a monitoring system capable of:

- Simulating realistic bearing degradation behavior
- Detecting abnormal vibration patterns in real time
- Communicating health status through industrial protocols
- Visualizing machine condition through a digital twin
- Supporting simple failure progression prediction

The project focuses on developing a scalable industrial cyber-physical monitoring system rather than only a standalone fault detection algorithm.

4 System Architecture and Methodology

4.1 System Architecture

The proposed system follows an edge-to-cloud Industrial Internet of Things (IIoT) architecture for bearing condition monitoring and fault prediction. The system integrates vibration data generation, edge anomaly detection, MQTT communication, and digital twin visualization into a cyber-physical monitoring platform.

The architecture consists of four functional layers:

Perception Layer

The perception layer represents the physical sensing level where bearing vibration signals are generated and acquired.

In this implementation, vibration signals are simulated to represent realistic bearing operating behavior including:

- Normal machine vibration
- Random noise disturbances
- Transient impact spikes
- Progressive degradation due to wear

These signals emulate data generated from an industrial vibration sensor.

Transport Layer

The transport layer is responsible for reliable communication between distributed components. The system uses MQTT publish-subscribe communication through the Eclipse Mosquitto broker.

Data topics used:

Sensor Data: sensors / group21 / bearing / data
Alert Data: alerts / group21 / bearing / status

MQTT provides lightweight and efficient industrial communication suitable for IIoT applications.

Edge Logic Layer

The edge logic layer performs:

- Real-time anomaly detection
- Health state classification
- Alert generation

Python edge intelligence subscribes to vibration data, processes signals and publishes fault alerts.

Application Layer

The application layer implements:

- Digital twin monitoring dashboard
- Vibration visualization
- Fault state monitoring

The dashboard is developed using Node-RED.

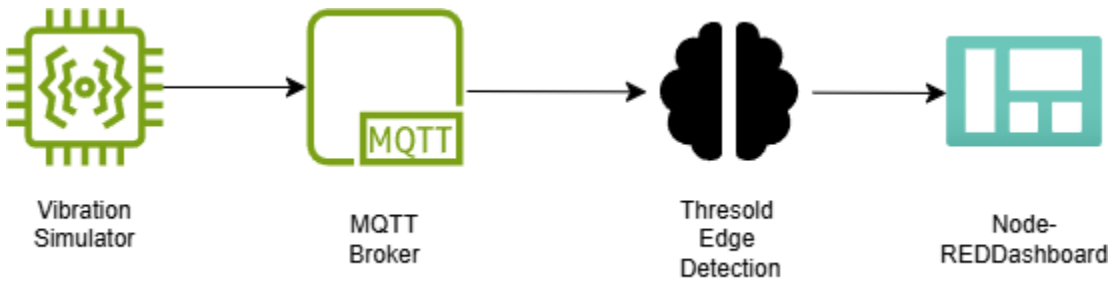

Figure 2: Edge-to-Cloud System Architecture for Bearing Failure Prediction

The system follows a publish–subscribe architecture using MQTT. The vibration simulator generates data, which is processed by the edge AI module and visualized through the Node-RED dashboard.

5 System Implementation

5.1 Dockerized Deployment

The proposed system is deployed using Docker Compose to simplify service orchestration, ensure modularity, and support reproducible deployment of industrial monitoring components.

The system consists of four containers as shown in Table 1.

Table 1: Dockerized Services Used in the System

Container	Function
mqtt	MQTT broker for message transport
node-red	Dashboard visualization and flow logic
python-publisher	Bearing vibration data simulation
python-ai	Edge anomaly detection

Docker Compose acts as the central orchestration layer, allowing all services to run simultaneously while maintaining isolation between components. This approach provides:

- Modular deployment of system components
- Container isolation for independent execution
- Scalability for future system expansion
- Reproducibility across different environments

5.2 Vibration Data Simulation

To emulate realistic bearing behavior, vibration signals are generated using a mathematical simulation model that incorporates periodic motion, noise, degradation, and fault events.

The vibration model is represented by: $Y = 2 + d + 0.8 \sin (t / 5) + n$

Where:

- y = simulated vibration amplitude
- d = degradation factor
- n = random noise components

The model consists of the following components:

- **Baseline vibration:** A sinusoidal function representing periodic motion in rotating machinery
- **Random noise:** Small fluctuations representing measurement uncertainty
- **Degradation factor:** A gradually increasing value representing bearing wear.

```
# Normal vibration: sinusoidal + noise
base_vibration = 2.0 + degradation
noise = random.uniform(-0.3, 0.3)
vibration = base_vibration + 0.8 * math.sin(t / 5) + noise
```

Figure 3: Simulated Bearing Vibration Model

The sinusoidal component represents normal machine vibration, while the degradation factor gradually increases the amplitude over time, simulating progressive bearing wear.

5.3 Fault Injection (Anomaly Simulation)

To simulate real-world faults, the system introduces random spikes into the vibration signal. These spikes represent sudden abnormal events such as bearing impacts or structural irregularities.

Fault events are generated with a small probability, allowing the system to test anomaly detection under realistic conditions.

```
# Occasionally inject a spike (bearing knock)
if random.random() < 0.04:
    vibration += random.uniform(2.0, 5.0)
```

Figure 4 : Simulated Bearing Fault Injection

These transient spikes create abnormal patterns in the vibration signal, which are later detected by the edge AI module.

5.4 Degradation Modeling

The system simulates long-term bearing wear by continuously increasing a degradation factor during operation: degradation += 0.002

This results in:

- gradual increase in vibration amplitude
- transition from normal to faulty behavior
- representation of progressive mechanical wear

```
# Slowly increase degradation over time (simulates wear)
degradation += 0.002
```

Figure 5 : Progressive Bearing Degradation

The gradual increase in vibration amplitude enables the system to model the full lifecycle of bearing wear and supports predictive maintenance analysis.

5.5 Simulated Data Output

The vibration simulator continuously generates data and publishes it to the MQTT broker in JSON format.

```
(base) hasininirmani@Hasinis-MacBook-Air edge-ai-group21 % docker logs -f python-publisher
{"vibration": 2.165, "degradation_level": 0.002, "timestamp": 1777625590.1043189}
{"vibration": 1.911, "degradation_level": 0.004, "timestamp": 1777625592.10557}
{"vibration": 2.594, "degradation_level": 0.006, "timestamp": 1777625594.1096299}
{"vibration": 2.259, "degradation_level": 0.008, "timestamp": 1777625596.111621}
{"vibration": 2.837, "degradation_level": 0.01, "timestamp": 1777625598.1125565}
{"vibration": 2.561, "degradation_level": 0.012, "timestamp": 1777625600.114063}
{"vibration": 3.044, "degradation_level": 0.014, "timestamp": 1777625602.1167638}
{"vibration": 2.543, "degradation_level": 0.016, "timestamp": 1777625604.1228716}
{"vibration": 2.691, "degradation_level": 0.018, "timestamp": 1777625606.1243281}
{"vibration": 2.785, "degradation_level": 0.02, "timestamp": 1777625608.1282911}
{"vibration": 2.742, "degradation_level": 0.022, "timestamp": 1777625610.132791}
{"vibration": 2.728, "degradation_level": 0.024, "timestamp": 1777625612.1348188}
{"vibration": 2.816, "degradation_level": 0.026, "timestamp": 1777625614.1403387}
{"vibration": 2.519, "degradation_level": 0.028, "timestamp": 1777625616.1446083}
{"vibration": 2.091, "degradation_level": 0.03, "timestamp": 1777625618.1476307}
{"vibration": 2.408, "degradation_level": 0.032, "timestamp": 1777625620.1495185}
{"vibration": 2.253, "degradation_level": 0.034, "timestamp": 1777625622.1549392}
{"vibration": 5.764, "degradation_level": 0.036, "timestamp": 1777625624.1611037}
{"vibration": 6.81, "degradation_level": 0.038, "timestamp": 1777625626.163631}
{"vibration": 1.462, "degradation_level": 0.04, "timestamp": 1777625628.1658583}
{"vibration": 1.275, "degradation_level": 0.042, "timestamp": 1777625630.1667793}
{"vibration": 1.091, "degradation_level": 0.044, "timestamp": 1777625632.1685476}
{"vibration": 1.428, "degradation_level": 0.046, "timestamp": 1777625634.1748123}
{"vibration": 1.144, "degradation_level": 0.048, "timestamp": 1777625636.1774843}
{"vibration": 1.209, "degradation_level": 0.05, "timestamp": 1777625638.1797276}
{"vibration": 1.105, "degradation_level": 0.052, "timestamp": 1777625640.1809542}
{"vibration": 1.362, "degradation_level": 0.054, "timestamp": 1777625642.1817807}
{"vibration": 1.383, "degradation_level": 0.056, "timestamp": 1777625644.183218}
{"vibration": 1.36, "degradation_level": 0.058, "timestamp": 1777625646.1856318}
{"vibration": 1.767, "degradation_level": 0.06, "timestamp": 1777625648.1901847}
{"vibration": 1.663, "degradation_level": 0.062, "timestamp": 1777625650.1918085}
{"vibration": 1.869, "degradation_level": 0.064, "timestamp": 1777625652.1937768}
{"vibration": 2.212, "degradation_level": 0.066, "timestamp": 1777625654.1949577}
{"vibration": 2.323, "degradation_level": 0.068, "timestamp": 1777625656.1976132}
```

Figure 6 :Sample Simulated Vibration Data

This data is transmitted to other system components for processing and visualization.

5.6 Edge AI Anomaly Detection

The edge AI module performs real-time anomaly detection using a hybrid approach combining statistical analysis and threshold-based classification.

Rolling Window Analysis

A rolling window of the last 20 vibration samples is maintained to compute statistical parameters.

Z-Score Calculation

The anomaly score is computed as: $z = (x-\mu) / \sigma$

Where:

- x = current vibration value
- μ = mean of the window
- σ = standard deviation

An anomaly is detected when: $z > 2.5$

Threshold-Based Classification

Condition	State
vibration ≤ 5	NORMAL
5 < vibration ≤ 7	WARNING
vibration > 7	CRITICAL

The edge AI module publishes the detected system status to the MQTT alert topic.

```
def detect_anomaly(value, history):
    if len(history) < WINDOW_SIZE:
        return False, "collecting"
    mean = np.mean(history)
    std  = np.std(history)
    # Z-score anomaly: more than 2.5 std deviations from mean
    z_score = (value - mean) / (std + 1e-6)
    if value > 7.0:
        return True, "CRITICAL - bearing failure imminent"
    elif value > 5.0 or z_score > 2.5:
        return True, "WARNING - abnormal vibration"
    return False, "NORMAL"
```

Figure 7 :Edge AI Anomaly Detection Logic

This process enables real-time classification of machine health conditions.

5.7 MQTT Communication

The system uses the MQTT protocol for communication between distributed components, following a publish–subscribe architecture.

Sensor data topic: sensors/group21/bearing/data

Alert data topic: alerts/group21/bearing/status

Communication Process

- The Python publisher sends vibration data to the broker
- The edge AI module subscribes and processes data
- Alerts are published back to the broker
- Node-RED subscribes to both topics and updates the dashboard

```
mosquitto > mosquitto.conf
1    listener 1883
2    allow_anonymous true
```

Figure 8: MQTT Communication Setup

This architecture enables efficient, low-latency communication between system components.

5.8 Digital Twin Dashboard Implementation

A real-time digital twin dashboard is implemented using Node-RED to visualize system behavior.

Dashboard Features

- Real-time vibration trend chart
- Current vibration gauge
- System health status (Normal, Warning, Critical)

The dashboard subscribes to both sensor and alert topics to provide a complete view of machine condition.

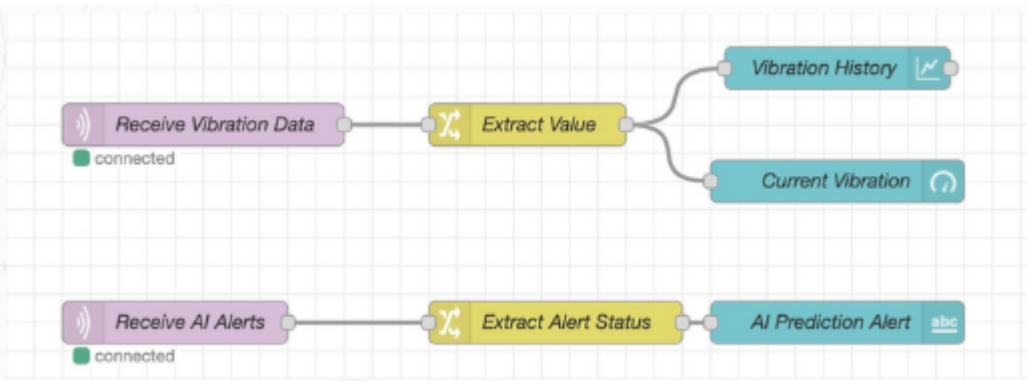

Figure 9: Node-RED Flow Configuration

This flow connects MQTT data streams to visualization components.

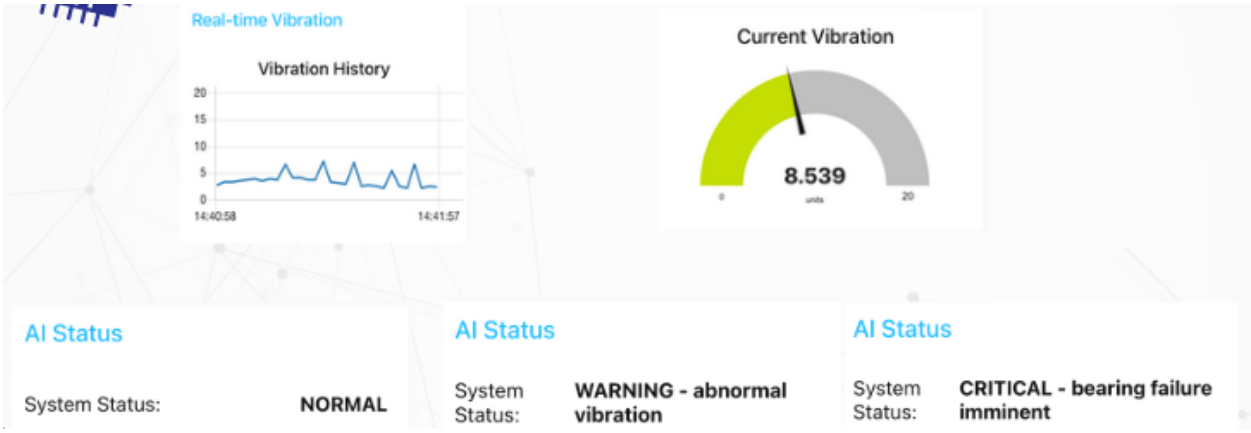

Figure 10: Digital Twin Dashboard Visualization

The dashboard provides an intuitive interface for monitoring vibration trends and detecting anomalies in real time.

6 Results and Discussion

6.1 Simulation Results

The developed system successfully simulated realistic bearing vibration behavior incorporating periodic motion, random noise, transient spikes, and progressive degradation.

The generated vibration signals exhibited:

- gradual increase in amplitude over time due to degradation
- random fluctuations representing environmental disturbances
- occasional high-magnitude spikes representing fault events

These characteristics closely resemble real-world bearing vibration patterns and provide a suitable dataset for anomaly detection.

6.2 Anomaly Detection Performance

The edge AI module effectively detected abnormal vibration patterns using a hybrid approach combining statistical analysis and threshold-based classification.

The system categorized machine conditions into three states:

Condition	System State
Normal vibration	NORMAL
Moderate abnormal vibration	WARNING
High vibration / fault	CRITICAL

The anomaly detection mechanism demonstrated:

- accurate detection of sudden spikes (fault events)
- early identification of abnormal trends
- stable classification during normal operation

The use of a rolling window ensured adaptive baseline estimation, improving detection reliability.

6.3 MQTT Communication Performance

The MQTT-based communication system successfully enabled real-time data transmission between system components.

Observed performance:

- low latency data transfer
- reliable message delivery
- efficient publish–subscribe communication

The broker effectively handled both sensor data and alert messages, ensuring synchronization between the edge AI module and the dashboard.

6.4 Dashboard Visualization

The Node-RED dashboard successfully visualized system behavior in real time.

Key features demonstrated:

- live vibration trend chart
- real-time gauge display
- system health status (Normal, Warning, Critical)

The dashboard acted as a digital twin, providing an intuitive interface for monitoring machine condition and identifying anomalies.

6.5 Discussion

The implemented system demonstrates how edge AI and Industrial IoT technologies can be combined to achieve predictive maintenance.

Although the system uses simulated data, the architecture is scalable and can be extended to real-world industrial applications with physical sensors.

The integration of edge intelligence, MQTT communication, and digital twin visualization reflects modern industrial monitoring practices.

7 Remaining Useful Life Estimation

Remaining Useful Life (RUL) estimation refers to predicting the time remaining before a system reaches failure conditions.

In this project, RUL estimation is based on the observed vibration degradation trend.

As the degradation factor increases, vibration amplitude gradually rises, moving the system through different operational states:

Normal → Warning → Critical

Failure is assumed when vibration exceeds the critical threshold:

Critical Threshold = 7

A simple RUL estimation can be expressed as:

RUL = Estimated Failure Time − Current Time

By analyzing the rate of increase in vibration, the system provides a basic indication of how close the bearing is to failure.

While this implementation is simplified, it demonstrates the concept of predictive maintenance and trend-based failure prediction.

8 Challenges Encountered

During the development of the system, several challenges were encountered:

8.1 Threshold Selection

Selecting appropriate threshold values for anomaly detection required balancing sensitivity and false alarm rates.

8.2 Simulation Realism

Designing a realistic vibration model required combining multiple factors such as noise, periodic motion, degradation, and fault spikes.

8.3 MQTT Integration

Ensuring reliable communication between multiple Docker containers required careful configuration of MQTT topics and services.

8.4 System Integration

Integrating simulation, edge AI, communication, and dashboard components into a unified system required debugging and synchronization of data flow.

9 Future Improvements

The system can be enhanced in several ways:

9.1 Hardware Implementation

Deploy the system using real sensors such as accelerometers connected to microcontrollers (e.g., ESP32).

9.2 Advanced Machine Learning

Replace threshold-based detection with advanced machine learning models such as:

- Autoencoders
- Neural networks
- Isolation Forest

9.3 Improved RUL Estimation

Implement more accurate prognostic models for predicting remaining useful life.

9.4 Industrial Dashboard Integration

Enhance visualization using:

- InfluxDB for data storage
- Grafana for advanced analytics

9.5 Real-Time Control Integration

Extend the system to include actuator control for automatic shutdown or maintenance alerts.

10 Conclusion

This project successfully developed an edge-to-cloud Industrial IoT system for bearing failure prediction using vibration simulation, edge AI anomaly detection, MQTT communication, and digital twin visualization.

The system demonstrated the ability to simulate realistic bearing behavior, detect anomalies in real time, and provide intuitive monitoring through a dashboard.

The use of Dockerized components ensured modular and scalable deployment, while the integration of edge intelligence and MQTT communication reflects modern industrial monitoring architectures.

Overall, the project highlights the effectiveness of combining edge AI and Industrial IoT technologies to support predictive maintenance and improve system reliability.

11 References

[1] MQTT Documentation, Eclipse Foundation.

[2] Node-RED Documentation, IBM/OpenJS Foundation.

[3] Docker Documentation.

[4] Eclipse Mosquitto Documentation.

[5] R. B. Randall, *Vibration-based Condition Monitoring*, Wiley, 2011.

[6] J. Lee et al., “Predictive Manufacturing Systems,” *Journal of Manufacturing Systems*, 2014.

[7] Z-score method for anomaly detection in time-series data.

12 Contribution of Group Members

Member Name	Registration Number	Contribution to Project	Contribution to Report
Supun Dhananji	E/20/062	Docker setup, MQTT broker configuration, system integration, Hardware device initial testing, platform io code setup	Docker deployment section, MQTT communication section
Hasini Nirmani	E/20/271	Node-RED dashboard design, UI visualization, simulation of vibration system implementation, logic implementation	Dashboard implementation section, Results section
Binuri Senavirathna	E/20/367	Edge AI anomaly detection (Z-score, thresholds), simulation of vibration system implementation, logic implementation	Edge AI section, RUL estimation section
Rashmika Sewmini	E/20/370	Vibration simulation model, fault injection, degradation modeling, Hardware device setup, platform io code setup testing	Simulation section, system implementation documentation